

Instead we should use something called dependency injection. Those familiar with the concept (Hey you java people!) will realise the additional functionalities and tremendous ease of use this offers to applications.

The solution is simple, instead of instantiating the `MyService` class, we directly pass it as a parameter in the `FirstComponent` class.

- `constructor FirstComponent (private myService : MyService)`

This way, any change in the `MyService` constructor has no effect on the existing code! Simple right?

Also, you are free to mock the `MyService` instance while unit testing `FirstComponent`, which is very helpful when you do not want to use a real service that is doing something slow (say a REST call for data).

Thus the refactored code would finally look something like this:

- `import { Injectable } from '@angular/core';`
- `import { MyService } from './service';`

```
export class FirstComponent{
  constructor FirstComponent (private myService : MyService)
  { }
  ngOnInit(){
  this.myService.serviceMethod();}
```

One important thing to note is that the `private` keyword in the constructor argument for the component actually defines a variable `myService` with the `private` modifier that we are free to use anywhere in the component (as seen under `ngOnInit()` function). We do not need to explicitly define a variable to hold the service instance.

All this is good but of course in real world situations services are long running and heavy duty tasks that are slow in nature. Using them directly in our components would not be a good idea as it would give an impression of a slow and sluggish application. This is where promises and observables come in.

Promises

A Promise is a proxy for a value not necessarily known when the promise is created. It allows you to associate handlers to an asynchronous action's eventual success value or failure reason. This lets asynchronous methods return values like synchronous methods: instead of the final value, the asynchronous method returns a promise for the value at some point in the future.